

数据结构与算法 实验报告

第三次



姓名 陈宇昂

班级 数学强基 2202 班

学号 2224110096

电话 **

Email yuang.chen2024@outlook.com

日期 2024-11-23

目录

任务 1: 实现 BST 数据结构	3
一、 题目	3
二、 数据结构设计	4
(一) BinNode<K, V> 类 (节点类)	4
(二) BST<K, V> 类 (二叉搜索树类)	4
三、 算法设计	4
四、 主干代码说明	5
(一) BinNode 类	5
(二) BST 类	5
(三) Test1 类	8
五、 运行结果展示	10
(一) Test1 测试结果	10
(二) 输出 output.txt	10
任务 2: 使用 BST 为文稿建立单词索引表	11
一、 题目	11
二、 数据结构设计	11
三、 算法设计	11
四、 主干代码说明	11
(一) BST 类	11
(二) Test1 类	12
五、 运行结果展示	13
六、 总结与收获	13
任务 3: 学会使用堆	14
一、 题目: 非递归快速排序	14
二、 数据结构设计	14
三、 算法设计	14
(一) insert(int element)	14
(二) delete()	14
(三) getMin()	15
(四) heapifyUp 方法	15
(五) heapifyDown 方法	15
(六) HeapSort (Heap heap, int[] array)	15
四、 主干代码说明	15
(一) Heap 类	15
(二) MinHeap 类	16
(三) TernaryHeap 类	16
(四) TestHeap 类	17
五、 运行结果展示	18
六、 总结与收获	18

任务 1：实现 BST 数据结构

一、题目

二叉检索树即 BST，是利用二叉树的非线性关系，结合数据之间的大小关系进行存储的一种用于检索目的的数据结构。一般情况下，对信息进行检索时，需要指定检索关键词（Key），根据该关键词找到所需要的信息（比如学生信息里关键词是学号，而姓名等信息就是该关键词对应的信息），所以当提到检索时，都会有“键值对”这个概念，用(key,value)表示键值和对应信息的关系。

下面的二叉检索树 ADT 在描述时，使用了模板类的方法，并且使用了两个模板参数 K 和 V（这表明 key 和 value 可以是不同类型的数据）。在后续的测试文件中，让 key 和 value 都是 String 类型，所以对模板参数运用有困难的同学，可以直接设定 key 和 value 的类型为 String。

注意：任务 2 中 BST 的 key 和 value 存放的数据类型可能不同。

图 1 为 BST 接口的定义（仅作参考），表 1 为对接口函数的详细说明。

```
public interface BST<K extends Comparable<K>, V> {
    public void insert(K key , V value);
    public V remove(K key);
    public V search(K key);
    public boolean update(K key , V value);
    public boolean isEmpty();
    public void clear();
    public void showStructure(PrintWriter pw) throws IOException;
    public void printInorder(PrintWriter pw) throws IOException;
}
```

图 1 BST 接口的定义

为了测试实现 BST 接口的数据结构是否运行正常，实验中提供两个文件，一个是 BST_testcases.txt，一个是 BST_result.txt 文件。其中，前一个包含了所有的对二叉检索树进行操作的命令内容，后一个则是执行命令文件之后的输出。具体的命令格式如表 2 所示。

表 2 命令的具体意义说明

命令符号	命令格式	命令举例
+	+(key , value)	+(auspices , "n.资助,赞助") 向二叉检索树中插入一个 key 为 auspices,value 为 "n.资助,赞助"的元素，插入的要求遵守 BST 接口中的 insert 方法的定义
-	-(key)	-(morass) 从二叉检索树中删除一个 key 为 morass 的元素，删除的要求遵守 BST 接口中的 remove 方法的定义
?	?(key)	?(hide) 在二叉检索树中查找一个 key 为 hide 的元素，查找的要求遵守 BST 接口中的 search 方法的定义
=	=(key, value)	=(laggard , "laggard") 将二叉检索中对 key 为 laggard 的元素的 value 值更新为 "laggard"，更新的要求遵守 BST 接口中的 update 方法的定义
#	#	# 调用二叉检索树的 showStructure 方法

图 2 给出了一个样例，用来说明命令的执行和对应输出的内容格式，供参考。

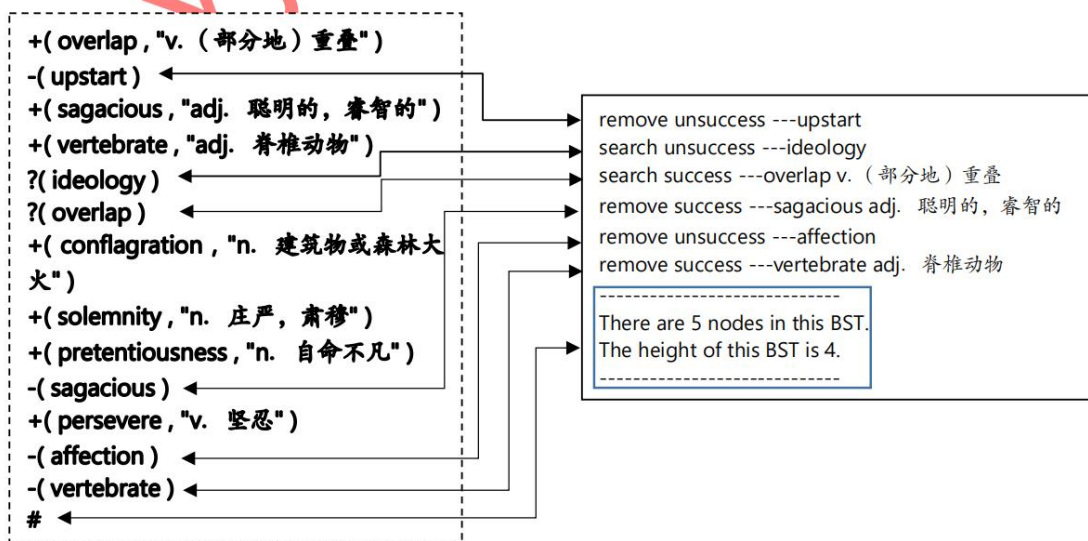


图 2 命令和输出格式对应关系

请大家要留意，本次输出的内容对每个命令的执行都有输出内容的要求（除了‘+’命令，因为该命令对应的 insert 方法没有返回值），而每个命令的输出格式很规律，结合每个命令对应的相应接口函数的返回值考虑即可。

图 2 样例中最后形成的二叉检索树的结点之间的关系如图 3 所示。

二、数据结构设计

（一）BinNode<K, V> 类（节点类）

这个类是 BST 的基础，代表了树中的一个节点。

K key: 每个节点有一个类型为 K 的键，这是二叉搜索树排序和搜索的基础。

V value: 与键相关联的值，类型为 V。

BinNode<K, V> left 和 BinNode<K, V> right: 分别指向该节点的左子节点和右子节点。提供了构造节点的方法，可以创建只包含键值的节点，或者是完整的节点（包含键、值以及左右子节点的链接）。还包含用于访问和修改节点数据的方法，例如获取和设置键、值、左右子节点等。

（二）BST<K, V> 类（二叉搜索树类）

这个类实现了二叉搜索树的主要功能。

BinNode<K, V> root: 树的根节点。还包含其他辅助字段，如节点数量等。

实现了插入（insert）、删除（remove）、搜索（search）和更新（update）等方法。

三、算法设计

（一）插入结点 insert()

insert(K key, V value):

在树中插入一个新节点。如果树中已存在相同键的节点，则更新该节点的值。

①从根节点开始，递归地比较待插入节点的键与当前节点的键。

②如果待插入节点的键小于当前节点，则向左子树递归；如果大于，则向右子树递归。

③如果找到相同键的节点，则更新节点值；否则，在适当的位置创建新节点。

（二）删除结点 remove()

remove(K key): 从树中删除键为 key 的节点。

- ①找到要删除的节点。
- ②如果节点是叶子节点，可以直接删除。
- ③如果节点有一个子节点，删除该节点，并用其子节点替代。
- ④如果节点有两个子节点，找到其右子树中的最小节点（或左子树中的最大节点），用这个节点替代要删除的节点，并删除这个最小（或最大）节点。

（三）搜索结点 search()

search(K key): 搜索键为 key 的节点。

- ①从根节点开始，递归地比较搜索键与当前节点的键。
- ②如果搜索键小于当前节点的键，则在左子树中搜索；如果大于，则在右子树中搜索。
- ③如果找到相同键的节点，返回该节点的值；否则，返回 null。

（四）更新结点 update()

与前文操作类似，如果最终为 null，表示未找到，返回 false；如果找到键值一致的结点，则更新它的 value。

（五）读写函数 readLine()

按行读入，根据字符串特征活用 split() 得到参数，并对树进行对应操作，在读到特定内容时停止。

四、主干代码说明

（一）BinNode 类

```
1 package task1and2;
2
3 public class BinNode <K extends Comparable<K>, V> { 33 个用法
4
5     //设置二叉树的单个节点
6     public K key;//键
7     public V value;//值
8     private BinNode<K,V> left;//左节点 4个用法
9     private BinNode<K,V> right;//右节点 4个用法
10    > public BinNode() { this(null,null); }
13    > public BinNode(K key, V value){ 3个用法
14        this.key = key;
15        this.value = value;
16    }
17    @ public BinNode(BinNode<K,V> p){ 1个用法
18        this.key = p.key();
19        this.value = p.value();
20        this.left = p.left();
21        this.right = p.right();
22    }
23    > public K key() { return key; }
26    > public V value() { return value; };
29    > public void setKey(K key) { this.key = key; }
32    > public void setValue(V value) { this.value = value; };
35    //新的修改值的方法
36    > public void setValue1(V value) { ((StringBuilder)(this.value)).append(" ").append(value); };
39
40    public BinNode<K,V> left(){ return left; }; 1个用法
41    public BinNode<K,V> setLeft(BinNode<K,V> p){return this.left = p; };
42
43    public BinNode<K,V> right(){ return right; }; 1个用法
44    public BinNode<K,V> setRight(BinNode<K,V> p){return this.right = p; };
45
46    public boolean isLeaf(){return left == null && right == null; };//判断是否是叶子节点
47 }
```

（二）BST 类

```
1 package task1and2;
2
3 import java.io.IOException;
4 import java.io.PrintWriter;
5
6 public class BST <K extends Comparable<K>, V>{ 6个用法
```

1、insert()

```

7      private BinNode<K,V> root;  15 个用法
8
9      private int numNode;//节点个数 8 个用法
10
11     //插入方法的实现
12     public void insert(K key, V value){ 2 个用法
13         assert(key!= null)&&(value!=null):"Wrong input";
14         root=inserthelp(root,key,value);
15     }
16
17     public BinNode<K, V> inserthelp(BinNode<K,V> rt, K key, V value){ 3 个用法
18         if(rt == null) {
19             numNode++;
20             return new BinNode<K,V>(key, value);
21         }
22         if(rt.key().compareTo(key) > 0)
23             rt.setLeft(inserthelp(rt.left(), key, value));
24         else if(rt.key().compareTo(key) < 0)
25             rt.setRight(inserthelp(rt.right(), key, value));
26         else updatehelp(rt, key, value);
27         return rt;
28     }

```

2、remove()

```

30     //删除方法的实现
31     public V remove(K key){
32         assert key != null : "Key is null.";
33         assert numNode != 0 : "Tree is empty.";
34         BinNode<K,V> temp = new BinNode<K,V>();
35         root = removehelp(root, key, temp);
36         return temp.value();
37     }
38
39     private BinNode<K,V> removehelp(BinNode<K,V> root, K key, BinNode<K,V> ans) { 3 个用法
40         if(root==null){
41             return null;
42         }
43         if(root.key().compareTo(key) > 0)//在左子树中递归操作
44             root.setLeft(removehelp(root.left(), key, ans));
45         else if(root.key().compareTo(key) < 0)//在右子树中递归操作
46             root.setRight(removehelp(root.right(), key, ans));
47         else {
48             //找到对应键值后的操作
49             ans.setValue(root.value);
50             if(root.left()==null){
51                 root=root.right();
52             } else if (root.right()==null) {
53                 root=root.left();
54             }else {
55                 BinNode<K,V> temp = getmin(root.right());
56                 root.setKey(temp.key());
57                 root.setValue(temp.value());
58                 root.setRight(deletemin(root.right()));
59             }
60             numNode--;
61         }
62         return root;
63     }

```

3、deletemin()

```

65 @    private BinNode<K,V> deletemin(BinNode<K,V> rt) { 2 个用法
66         if(rt.left() == null)
67             // 左树为空，没有更小的结点了，找到了这棵树的最小值结点，
68             // 返回它的右子树（它是null或是二叉树无影响）
69             return rt.right();
70         else{
71             // 继续进行寻找
72             rt.setLeft(deletemin(rt.left()));
73             return rt;
74         }
75     }

```

4、getmin()

```

77 @ private BinNode<K, V> getmin(BinNode<K, V> rt) { 2个用法
78     if(rt.left() == null)
79         // 左树为空，没有更小的结点了
80         return new BinNode<K,V>(rt);
81     else return getmin(rt.left()); // 继续寻找
82 }

```

5、search()

```

84 // 查找
85 public V search(K key){
86     assert key != null : "Key is null.";
87     return searchhelp(root, key);
88 }
89
90 private V searchhelp(BinNode<K,V> root, K key) { 3个用法
91     if(root == null) return null;
92     if(root.key().compareTo(key) > 0)
93     return searchhelp(root.left(), key);
94     else if(root.key().compareTo(key) < 0)
95     return searchhelp(root.right(), key);
96     else
97     return root.value();
98 }

```

6、update()

```

100 // 替换方法的实现
101 public boolean update(K key, V value){ 1个用法
102     assert (key != null) && (value != null) : "Wrong input.";
103     return updatehelp(root, key, value);
104 }
105 private boolean updatehelp(BinNode<K,V> root, K key, V value) { 4个用法
106     if(root==null){
107         return false;
108     }
109     if(root.key().compareTo(key)>0){
110     return updatehelp(root.left(), key, value);
111     }else if(root.key().compareTo(key)<0){
112     return updatehelp(root.right(), key, value);
113     }else {
114         root.setValue(value);
115     }
116     return true;
117 }

```

7、isEmpty()、clear()、getHeight()

```

120 > public boolean isEmpty() { return root == null; };
123 v public void clear(){
124     numNode = 0;
125     root = null;
126 }
127 v public int getHeight(BinNode<K,V> rt){ 5个用法
128     if(rt==null){
129         return 0;
130     }else {
131         int left;
132         int right;
133     left=getHeight(rt.left());
134     right=getHeight(rt.right());
135     return Math.max(left,right)+1;
136     }
137 }

```

8、showStructure1()、showStructure()

```

140     public void showStructure1(){ 0个用法
141         System.out.println("There are " + numNode + " nodes in this BST.");
142         System.out.println("The height of this BST is " + getHeight(root) + ".");
143     }
144 @    public void showStructure(PrintWriter pw) throws IOException { 1个用法
145         pw.println("-----");
146         pw.println("There are " + numNode + " nodes in this BST.");
147         pw.println("The height of this BST is " + getHeight(root) + ".");
148         pw.println("-----");
149         System.out.println("-----");
150         System.out.println("There are " + numNode + " nodes in this BST.");
151         System.out.println("The height of this BST is " + getHeight(root) + ".");
152         System.out.println("-----");
153     }

```

9、printInorder1()、printhepl1()、printInorder()、printhepl()

```

154 >    public void printInorder1() { printhepl1(root); }
157    private void printhepl1(BinNode<K,V> rt){ 3个用法
158        if(rt == null) return;
159        printhepl1(rt.left());
160        System.out.print("[");
161        System.out.print(rt.key());
162        System.out.print(" --- < ");
163        System.out.print(rt.value());
164        System.out.println(">]");
165        printhepl1(rt.right());
166    }
167    public void printInorder(PrintWriter pw) throws IOException{ 1个用法
168        printhepl(root, pw);
169    }
170    private void printhepl(BinNode<K,V> rt, PrintWriter pw) throws IOException{ 3个用法
171        if(rt == null) return;
172        printhepl(rt.left(), pw);
173        pw.print("[");
174        pw.print(rt.key());
175        pw.print(" --- < ");
176        pw.print(rt.value());
177        pw.println(">]");
178        printhepl(rt.right(), pw);
179    }

```

(三) Test1 类

1、readline()

```

13    public static void readline(BST<String, String> bst, PrintWriter pw, File file) throws IOException { 1个用法
14        Scanner in = new Scanner(file);
15        String value = new String();
16        String temp = new String();
17        while(!temp.equals("stop")){
18            temp = in.nextLine();
19            switch (temp.charAt(0)) {
20                case '+' -> {
21                    if (temp.split(" ")[2].equals("")) {
22                        bst.insert(getKey(temp), "");
23                        break;
24                    }
25                    bst.insert(getKey(temp), getValue(temp));
26                }
27                case '-' -> {
28                    if((value = bst.remove(getKey(temp))) == null) {
29                        pw.println("remove unsuccess ---" + getKey(temp));
30                        System.out.println("remove unsuccess ---" + getKey(temp));
31                        break;
32                    }
33                    value = value.equals("") ? "" : " " + value;
34                    pw.println("remove success ---" + getKey(temp) + value);
35                    System.out.println("remove success ---" + getKey(temp) + value);
36                }

```

```

37         case '?' -> {
38             if((value = bst.search(getKey(temp))) == null) {
39                 pw.println("search unsuccess ---" + getKey(temp));
40                 System.out.println("search unsuccess ---" + getKey(temp));
41                 break;
42             }
43             value = value.equals("") ? "" : " " + value;
44             pw.println("search success ---" + getKey(temp) + value);
45             System.out.println("search success ---" + getKey(temp) + value);
46         }
47         case '=' -> {
48             if(!bst.update(getKey(temp), getValue(temp))) {
49                 pw.println("update unsuccess ---" + getKey(temp));
50                 System.out.println("update unsuccess ---" + getKey(temp));
51                 break;
52             }
53             value = getValue(temp);
54             pw.println("update success ---" + getKey(temp) + " " + value);
55             System.out.println("update success ---" + getKey(temp) + " " + value);
56         }
57         case '#' -> bst.showStructure(pw);
58     }
59 }
60 in.close();
61 }

```

2、compareFiles()

```

63 private static void compareFiles(String file1, String file2) throws IOException { 1个用法
64     BufferedReader br1 = new BufferedReader(new FileReader(file1));
65     BufferedReader br2 = new BufferedReader(new FileReader(file2));
66
67     String line1 = br1.readLine();
68     String line2 = br2.readLine();
69
70     boolean areEqual = true;
71     int lineNum = 1;
72
73     while (line1 != null || line2 != null) {
74         if(line1 == null || line2 == null || !line1.equals(line2)) {
75             areEqual = false;
76             break;
77         }
78         line1 = br1.readLine();
79         line2 = br2.readLine();
80         lineNum++;
81     }
82
83     if (areEqual) {
84         System.out.println("两个文件的内容相同。");
85     } else {
86         System.out.println("两个文件内容不同。它们在第 " + lineNum + " 行不同");
87         System.out.println("File1 在第 " + lineNum + " 行的内容为 " +
88             line1 + ", File2 在第 " + lineNum + " 行的内容是 " + line2);
89     }
90
91     br1.close();
92     br2.close();
93 }

```

3、main()

```

112 public static void main(String[] args) throws IOException {
113     // 第一题
114     String filename="D:\\MyJavaProject\\Graph_lab2\\src\\homework3_testcases.txt";
115     // 保存 PrintWriter 输出的文件
116     String outputFilename = "D:\\MyJavaProject\\Graph_lab2\\src\\output.txt";
117     // 待比较的结果文件
118     String resultFilename = "D:\\MyJavaProject\\Graph_lab2\\src\\homework3_result.txt";
119
120     BST<String,String> bst = new BST<String,String>();
121     PrintWriter pw = new PrintWriter(new FileWriter(outputFilename));
122     readLine(bst, pw,new File(filename));
123     pw.close();
124     // 比较两个文件的内容
125     compareFiles(outputFilename, resultFilename);
126     pw.close();

```

五、运行结果展示

(一) Test1 测试结果

```

-----
There are 1 nodes in this BST.
The height of this BST is 1.
-----
remove unsuccessful ---vertebrate
update success ---overlap overlap
remove unsuccessful ---affection
search success ---persevere v. 坚忍
-----
There are 11 nodes in this BST.
The height of this BST is 5.
-----
update success ---pretentiousness pretentiousness
-----
There are 13 nodes in this BST.
The height of this BST is 5.
-----
remove success ---sill n. 基石, 门槛, 窗台
search unsuccessful ---listless
search success ---gollop v. n. 大口吞咽
update success ---sagacious sagacious
remove unsuccessful ---propitiate
update success ---ideology ideology
search success ---puerile adj. 幼稚的, 儿童的
update success ---laggard laggard
search unsuccessful ---penalize
search success ---deify v. 奉为神, 崇拜
search unsuccessful ---midget
search success ---mutton n. 羊肉
update success ---conflagration conflagration
search unsuccessful ---morass
search success ---breach v. 毁坏, 泄密n. 缺口
search unsuccessful ---crudity
search success ---icon n. 圣像, 偶像
-----
There are 412 nodes in this BST.
The height of this BST is 17.
-----
update success ---orientation orientation
update success ---credence credence
search success ---zephyr n. 和风, 西风
remove success ---supplicate v. 恳求, 乞求
search success ---dynamite n. 炸药, 扣人心弦的事
remove unsuccessful ---scurrilous
remove unsuccessful ---regiment
search unsuccessful ---ply
search success ---ellipsis n. 省略
-----
There are 417 nodes in this BST.
The height of this BST is 17.
-----
remove unsuccessful ---arsonist
search unsuccessful ---vandalism
search success ---royalty n. 皇室, 版税 (付给著作人的钱)
-----
There are 424 nodes in this BST.
The height of this BST is 17.
-----
remove success ---gambol n. 雀跃, 嬉戏
remove unsuccessful ---doting
两个文件的内容相同。
Process finished with exit code 0

```

BST 各个方法均实现成功。

(二) 输出 output.txt

```

1 -----
2 There are 1 nodes in this BST.
3 The height of this BST is 1.
4 -----
5 remove unsuccessful ---vertebrate
6 update success ---overlap overlap
7 remove unsuccessful ---affection
8 search success ---persevere v. 坚忍
9 -----
10 There are 11 nodes in this BST.
11 The height of this BST is 5.
12 -----
13 update success ---pretentiousness pretentiousness
14 -----
15 There are 13 nodes in this BST.
16 The height of this BST is 5.
17 -----
18 remove success ---sill n. 基石, 门槛, 窗台
19 search unsuccessful ---listless
20 search success ---gollop v. n. 大口吞咽
21 update success ---sagacious sagacious
22 remove unsuccessful ---propitiate
23 update success ---ideology ideology
24 search success ---puerile adj. 幼稚的, 儿童的
25 update success ---laggard laggard
26 search unsuccessful ---penalize
27 search success ---deify v. 奉为神, 崇拜
28 search unsuccessful ---midget
29 search success ---mutton n. 羊肉
30 update success ---conflagration conflagration
31 search unsuccessful ---morass
32 search success ---breach v. 毁坏, 泄密n. 缺口
33 search success ---sagacious sagacious
34 remove success ---miscreant n. 恶棍, 歹徒
35 -----
36 There are 33 nodes in this BST.
37 The height of this BST is 8.
38 -----
39 search unsuccessful ---guarded

```

output.txt 节选展示

任务 2：使用 BST 为文稿建立单词索引表

一、题目

在使用很多文字编辑软件时，都提供了对所编辑的文稿进行搜索的功能。当文稿内容的数据量比较大时，检索的速度就必须要进行考虑了。利用任务 1 中构建的 BST 数据结构，可以有效地解决这个问题。将文稿中出现的每个单词都插入到用 BST 构建的搜索表中，记录每个单词在文章中出现的行号。

实验材料里有一篇英文文稿，文件名为 article.txt¹，请按照上面的样例根据该 article.txt 中的内容构建索引表，并将索引表的内容按照单词的字典序列输出到 index_result.txt 文件中。

在构建 BST 中的结点数据时，请认真思考记录行号的数据类型，如果可能，尽可能使用在之前实验中自己构建的数据结构，也是对之前数据结构使用的一种验证。

二、数据结构设计

沿用任务 1 的 BST 数据结构。将原 BST 中键值对的类型设置改为 String 与 StringBuilder，方便对新生成的行数进行添加。并同时更新 insert1()、update1() 以及 setValue() 函数。

三、算法设计

（一）更新函数 update1()

将原来的 update() 方法的 setValue() 方法改为 setValue1()，从覆盖值改为后接值即可。

（二）读写函数 index()

要处理的是小说文本，参杂了多种标点符号。为了提取出准确的单词，按行循环接受文本，利用 split() 函数和正则表达式的知识清除冗余的标点符号得到文本中的单词，并将之连同行号插入新的 BST 树中。

四、主干代码说明

（一）BST 类

1、update1()

```
201 //新替换方法的实现
202 private boolean updatehelp1(BinNode<K,V> root, K key, V value) { 3个用法
203     if(root==null){
204         return false;
205     }
206     if(root.key().compareTo(key)>0){
207         return updatehelp1(root.left(), key, value);
208     }else if(root.key().compareTo(key)<0){
209         return updatehelp1(root.right(), key, value);
210     }else {
211         root.setValue1(value);
212     }
213     return true;
214 }
```

2、insert1()

```
182 //新插入方法的实现
183 public void insert1(K key, V value){ 1个用法
184     assert(key!= null)&&(value!=null):"Wrong input";
185     root=inserthelp1(root,key,value);
186 }
```

3、inserthelp1()

```

188     public BinNode<K, V> inserthelp1(BinNode<K,V> rt, K key, V value){ 3个用法
189         if(rt == null) {
190             numNode++;
191             return new BinNode<K,V>(key, value);
192         }
193         if(rt.key().compareTo(key) > 0)
194             rt.setLeft(inserthelp1(rt.left(), key, value));
195         else if(rt.key().compareTo(key) < 0)
196             rt.setRight(inserthelp1(rt.right(), key, value));
197         else updatehelp1(rt, key, value);
198         return rt;
199     }

```

(二) Test1 类

1、index()

```

95     public static void index(BST bst, PrintWriter pw, File file) throws IOException { 1个用法
96         Scanner in = new Scanner(file);
97         String temp = in.nextLine();
98         int line = 0;
99         while(!temp.equals("stop")){
100             for(String str : temp.split(regex: "[\\n,.?!\"';()*\\-]|--|\\s+")) {
101                 if ("".equals(str) || str.length() < 3)
102                     continue;
103                 bst.insert1(str, new StringBuilder(line + ""));
104             }
105             temp = in.nextLine();
106             line++;
107         }
108         bst.printInorder(pw);
109         in.close();
110     }

```

2、main()

```

112     public static void main(String[] args) throws IOException {
130         String filename="task1and2\\BST_testcases.txt";
131         String outputFilename = "task1and2\\output.txt";
132         String resultFilename = "task1and2\\BST_result.txt";
133
134         BST<String,String> bst = new BST<String,String>();
135         PrintWriter pw = new PrintWriter(new FileWriter(outputFilename));
136         readLine(bst, pw,new File(filename));
137
138         pw.close();
139         compareFiles(outputFilename, resultFilename);
140
141         BST bstindex = new BST();
142         File file = new File( pathname: "task1and2\\article.txt");
143         PrintWriter pwi = new PrintWriter( fileName: "task1and2\\index_result.txt");
144
145         index(bstindex, pwi, file);
146
147         pw.close();
148         pwi.close();
149     }

```

五、运行结果展示

1	[000 --- < 11448 11527 11530 13817 14621 15730 16576 19621 19679 19927 20347 20353 20560 >]
2	[100 --- < 16970 >]
3	[1000 --- < 20565 22579 >]
4	[10s --- < 14701 17778 >]
5	[10th --- < 13999 >]
6	[117 --- < 16090 16095 >]
7	[117th --- < 27146 27588 >]
8	[126b --- < 23929 23971 23984 >]
9	[129 --- < 1986 >]
10	[12s --- < 16103 >]
11	[12th --- < 14000 >]
12	[140 --- < 13362 >]
13	[14th --- < 26115 >]
14	[150 --- < 12759 >]
15	[1607 --- < 25766 >]
16	[1642 --- < 1675 >]
17	[16A --- < 17635 >]
18	[1840 --- < 32607 >]
19	[1846 --- < 18596 >]
20	[1855 --- < 24955 >]
21	[1857 --- < 5010 >]
22	[1858 --- < 9893 >]
23	[1860 --- < 4000 >]
24	[1869 --- < 13711 13992 14205 >]
25	[1870 --- < 13711 >]
26	[1871 --- < 5011 >]
27	[1875 --- < 28592 >]
28	[1878 --- < 55 5276 5293 13727 >]
29	[1881 --- < 32610 >]
30	[1882 --- < 5313 5463 5794 >]
31	[1883 --- < 13742 13824 >]
32	[1884 --- < 14689 32611 >]

14798	[yellow --- < 986 1140 1289 5205 5542 5623 5624 5642 5649 6033 7274 8187 8206 8279 8975 11583 12979 13123 14417 14631 16126 16902 17370 174
14799	[yellowish --- < 916 >]
14799	[yells --- < 8882 >]
14799	[yelp --- < 7131 >]
14799	[yelps --- < 6915 >]
14799	[yes --- < 1617 2723 5348 6056 6290 6986 9895 11232 11882 11972 13091 14064 15193 15820 15974 16022 16526 17188 17248 17320 18384 18561 186
14799	[yesterday --- < 1674 2269 2386 4865 4872 4879 6939 7227 7573 7728 11153 12285 13920 15434 18687 18693 20378 20548 21349 21353 21764 21771
14799	[westernnight --- < 7250 >]
14799	[yet --- < 147 171 218 433 578 886 898 1096 1170 2189 2429 2430 2659 2785 2835 3146 3300 3305 3448 3619 3949 3976 4140 4931 5072 5283 5312
14799	[yield --- < 6223 >]
14800	[yoked --- < 2967 >]
14801	[yonder --- < 8163 8181 16329 22318 23103 25805 28536 32333 >]
14802	[you --- < 118 126 132 147 164 177 187 189 194 202 218 221 243 247 250 257 273 321 322 322 323 333 340 349 357 377 378 382 382 479 480 489
14803	[you: --- < 20875 >]
14804	[young --- < 109 559 1886 2192 2521 2881 2887 2934 3081 3087 3163 3172 3186 3204 3228 3277 3393 3398 3404 3438 3494 3501 3532 3544 3562 357
14806	[younger --- < 4155 4828 11884 18703 21110 24982 26692 26720 28514 28536 28982 29862 >]
14806	[youngest --- < 2863 >]
14807	[youngster --- < 4598 13726 19781 32698 33190 >]
14808	[youngsters --- < 1919 >]
14809	[your --- < 194 221 349 636 656 668 677 712 760 846 881 901 949 962 1102 1165 1170 1209 1237 1384 1398 1508 1581 1513 1611 1666 1666 1724 1
14810	[yours --- < 173 1499 6052 6063 6776 7914 8477 8855 9252 9680 10624 10694 11016 11194 12331 13174 13608 14570 16206 16328 18790 19043 21225
14811	[yourself --- < 118 957 1584 1635 2730 2791 3170 3427 3469 4700 4849 4970 5334 6176 6434 6605 6897 7255 7656 7932 9649 9937 10262 10810 108
14812	[yourselves --- < 5325 7448 10934 11549 16375 16485 23495 27635 >]
14813	[youth --- < 2963 3123 3504 5431 5652 7211 10505 10844 10846 13034 15353 15833 19706 23630 24573 26837 27907 31819 31891 32974 >]
14814	[youths --- < 1908 >]
14815	[zeal --- < 454 24050 >]
14816	[zealous --- < 21808 >]
14817	[zero --- < 14090 20664 >]
14818	[zest --- < 16956 >]
14819	[zigzag --- < 7101 15997 >]
14820	[zum --- < 9521 9521 >]

输出文件 index_result.txt 的内容节选。

六、总结与收获

通过本次实验，我加深了对数据结构和算法设计的理解，特别是二叉搜索树的构建及其在各种应用场景中的用法。首先，我清晰地认识到 BST 在不同查询需求下的独特优势与适用情境。例如，BST 能够在对数时间复杂度内完成有序数据的搜索、插入和删除操作，这对于需要频繁查询的大型数据集而言极为高效。这使我深刻体会到，在面对具体问题时，精准选择或设计最适合的数据结构，对于提升程序性能和资源利用率至关重要。

任务 3：学会使用堆

一、题目：非递归快速排序

堆是一棵完全二叉树，是二叉树众多应用中一个最适合用数组方式存储的树形，所以必须要掌握。堆的主要用途是构建优先队列 ADT，除此之外，高效的从若干个数据中连续找到剩余元素中最小（或最大）都是堆的应用场景，比如构建 Huffman 树时，我们需要从候选频率中选择最小的两个；比如最短路径 Dijkstra 算法中要从最短路径估计值数组中选取当前最小值等，所有的这些应用都因为使用了堆而如虎添翼。下面从所列的几个方面对堆进行实践：

- 1) 参看书中的代码，撰写一个具有 insert、delete、getMin 等方法的 Min_Heap
- 2) 完成一个使用 1) 编写的 Min_Heap 的排序算法；
- 3) 上课讲解的堆是二叉树的一个应用，因此也称为二叉堆。如果将堆的概念扩展到三叉树

（树中每个结点的子结点数最多是 3 个），此时将形成三叉堆。除了树形和二叉堆不一致外，堆序的要求是完全一致的。使用完全三叉树形成堆，在 n 个结点下显然会降低整棵树的高度，但是在维持某个结点的堆序时需要比较的次数会增加（这个时候会是 3 个子结点之间进行互相比求最大或最小），这也是一个权衡问题。现在要求将 1) 中实现的二叉堆改造成三叉堆，并通过 2) 中实现的排序验证这个三叉堆是否正确。

二、数据结构设计

采用一个基类 Heap 定义堆的一些操作，其中有两个抽象方法由其子类实现，用以后续拓展 K 叉堆。这样做目的是定义一个通用堆的框架，包含堆的基本属性和方法。具体如下：

int[] heap: 存储堆元素的数组。

int size: 当前堆中元素的数量。

int capacity: 堆的最大容量。

protected abstract void heapifyUp(int index): 上浮调整，由子类实现。

protected abstract void heapifyDown(int index): 下沉调整，由子类实现。

第一个子任务要求实现的三个方法：

public void insert(int element): 向堆中插入元素。

public int delete(): 从堆中删除并返回最小元素。

public int getMin(): 获取堆顶元素。

public int getSize(): 获取堆的大小。

protected void swap(int i, int j): 交换堆中元素

第二个子任务实现了子类 MinHeap 和 TernaryHeap，分别实现二叉堆和三叉堆的具体操作。其中二叉堆和三叉堆重写了 heapifyUp 和 heapifyDown 方法以维护各自的堆属性。

在完成了数据结构的定义后在 HeapSort 类中完成了对于堆排序的实现，满足题目中：“完成一个使用 1) 编写的 Min_Heap 的排序算法；”的要求。最后设计了 HeapTest 类来完成分别对二叉堆 MinHeap 和三叉堆 TernaryHeap 进行测试。

三、算法设计

（一）insert(int element)

算法：将新元素添加到数组末尾，然后使用 heapifyUp 方法调整堆。

（二）delete()

算法：移除并返回数组首元素（堆顶），将末尾元素移至堆顶，然后使用 heapifyDown 方法调整堆。

（三）getMin()

算法：直接返回数组首元素。

（四）heapifyUp 方法

在插入新元素后调整堆，确保维护堆的性质。

算法：比较新插入的节点与其父节点的值，如果不满足堆的性质，则交换它们的位置，递归地继续这一过程直到恢复堆的性质。

（五）heapifyDown 方法

在删除元素后调整堆，确保维护堆的性质。

算法：将根节点与其子节点中的最小值进行比较，如果根节点不是最小值，则与最小的子节点交换，递归地继续这一过程直到恢复堆的性质。

（六）HeapSort (Heap heap, int[] array)

算法：将传入的数组逐个按规则放入堆中，再依次去除堆顶元素。

四、主干代码说明

（一）Heap 类

```

1  package task3;
2
3  public abstract class Heap { 4个用法 2个继承者
4      protected int[] heap; 24个用法
5      protected int size;
6      protected int capacity; 2个用法
7
8      public Heap(int capacity) { 1个用法
9          this.capacity = capacity;
10         this.heap = new int[capacity];
11         this.size = 0;
12     }
13
14     protected abstract void heapifyUp(int index); 3个用法 2个实现
15     protected abstract void heapifyDown(int index); 3个用法 2个实现
16
17     protected void swap(int i, int j) { 4个用法
18         int temp = heap[i];
19         heap[i] = heap[j];
20         heap[j] = temp;
21     }
22
23     public void insert(int element) { 8个用法
24         if (size >= capacity) {
25             throw new IllegalStateException("Heap is full");
26         }
27         heap[size] = element;
28         heapifyUp(size);
29         size++;
30     }
31
32     public int delete() { 3个用法
33         if (size == 0) {
34             throw new IllegalStateException("Heap is empty");
35         }
36         int minElement = heap[0];
37         heap[0] = heap[size - 1];
38         size--;
39         heapifyDown(index: 0);
40         return minElement;
41     }

```

```

43 > public int getSize() { return size; }
44
45
46
47 public int getMin() { 1个用法
48     if (size == 0) {
49         throw new IllegalStateException("Heap is empty");
50     }
51     return heap[0];
52 }
53
54 public class HeapSort { 1个用法
55
56 @ public static void sort(Heap heap, int[] array) { 1个用法
57     for (int i = 0; i < array.length; i++) {
58         heap.insert(array[i]);
59     }
60     for (int i = 0; i < array.length; i++) {
61         array[i] = heap.delete();
62     }
63 }
64 }
65
66 }

```

(二) MinHeap 类

```

1 package task3;
2
3 @ public class MinHeap extends Heap { 2个用法 1个继承者
4
5     public MinHeap(int capacity) { 2个用法
6         super(capacity);
7     }
8
9     @Override 3个用法 1个重写
10    protected void heapifyUp(int index) {
11        int parentIdx = (index - 1) / 2;
12        if (index > 0 && heap[parentIdx] > heap[index]) {
13            swap(parentIdx, index);
14            heapifyUp(parentIdx);
15        }
16    }
17
18    @Override 3个用法 1个重写
19    protected void heapifyDown(int index) {
20        int smallest = index;
21        int leftChildIdx = 2 * index + 1;
22        int rightChildIdx = 2 * index + 2;
23
24        if (leftChildIdx < size && heap[leftChildIdx] < heap[smallest]) {
25            smallest = leftChildIdx;
26        }
27        if (rightChildIdx < size && heap[rightChildIdx] < heap[smallest]) {
28            smallest = rightChildIdx;
29        }
30        if (smallest != index) {
31            swap(index, smallest);
32            heapifyDown(smallest);
33        }
34    }
35 }

```

(三) TernaryHeap 类

```

1 package task3;
2
3 public class TernaryHeap extends MinHeap { 1个用法
4 >     public TernaryHeap(int capacity) { super(capacity); }
5
6
7

```

```

8      @Override 3个用法
9      protected void heapifyUp(int index) {
10         int parentIdx = (index - 1) / 3;
11         if (index > 0 && heap[parentIdx] > heap[index]) {
12             swap(parentIdx, index);
13             heapifyUp(parentIdx);
14         }
15     }
16
17     @Override 3个用法
18     protected void heapifyDown(int index) {
19         int smallest = index;
20         int childIndex1 = 3 * index + 1;
21         int childIndex2 = 3 * index + 2;
22         int childIndex3 = 3 * index + 3;
23
24         if (childIndex1 < size && heap[childIndex1] < heap[smallest]) {
25             smallest = childIndex1;
26         }
27         if (childIndex2 < size && heap[childIndex2] < heap[smallest]) {
28             smallest = childIndex2;
29         }
30         if (childIndex3 < size && heap[childIndex3] < heap[smallest]) {
31             smallest = childIndex3;
32         }
33
34         if (smallest != index) {
35             swap(index, smallest);
36             heapifyDown(smallest);
37         }
38     }
39 }

```

(四) TestHeap 类

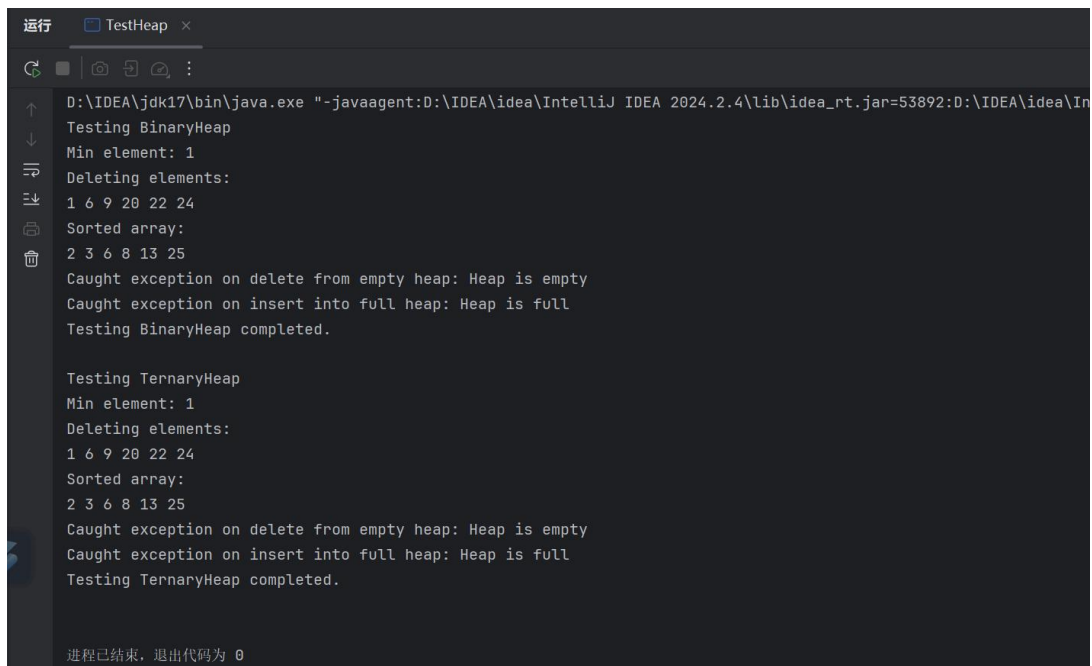
```

1  package task3;
2
3  public class TestHeap {
4      public static void main(String[] args) {
5          testHeap(new MinHeap( capacity: 10), heapType: "BinaryHeap");
6          testHeap(new TernaryHeap( capacity: 10), heapType: "TernaryHeap");
7      }
8
9      @ private static void testHeap(Heap heap, String heapType) { 2个用法
10         System.out.println("Testing " + heapType);
11
12         heap.insert( element: 22);
13         heap.insert( element: 1);
14         heap.insert( element: 6);
15         heap.insert( element: 20);
16         heap.insert( element: 24);
17         heap.insert( element: 9);
18
19         System.out.println("Min element: " + heap.getMin());
20
21         System.out.println("Deleting elements:");
22         while (heap.getSize() > 0) {
23             System.out.print(heap.delete() + " ");
24         }
25         System.out.println();
26
27         int[] array = {25, 3, 13, 8, 6, 2};
28         Heap.HeapSort.sort(heap, array);
29         System.out.println("Sorted array:");
30         for (int value : array) {
31             System.out.print(value + " ");
32         }
33         System.out.println();

```

```
35         try {
36             heap.delete();
37         } catch (IllegalStateException e) {
38             System.out.println("Caught exception on delete from empty heap: " + e.getMessage());
39         }
40
41         try {
42             for (int i = 0; i < 11; i++) {
43                 heap.insert(i);
44             }
45         } catch (IllegalStateException e) {
46             System.out.println("Caught exception on insert into full heap: " + e.getMessage());
47         }
48
49         System.out.println("Testing " + heapType + " completed.\n");
50     }
51 }
```

五、运行结果展示



```
运行 TestHeap x
D:\IDEA\jdk17\bin\java.exe "-javaagent:D:\IDEA\idea\IntelliJ IDEA 2024.2.4\lib\idea_rt.jar=53892:D:\IDEA\idea\In
Testing BinaryHeap
Min element: 1
Deleting elements:
1 6 9 20 22 24
Sorted array:
2 3 6 8 13 25
Caught exception on delete from empty heap: Heap is empty
Caught exception on insert into full heap: Heap is full
Testing BinaryHeap completed.

Testing TernaryHeap
Min element: 1
Deleting elements:
1 6 9 20 22 24
Sorted array:
2 3 6 8 13 25
Caught exception on delete from empty heap: Heap is empty
Caught exception on insert into full heap: Heap is full
Testing TernaryHeap completed.

进程已结束，退出代码为 0
```

说明二叉堆三叉堆的实现完全正确

六、总结与收获

在本任务中，通过对二叉堆与三叉堆实现方法的探索，我深刻体验到了这两种数据结构在数据管理中的高效性，它们不仅能在插入与删除操作中稳定保持堆的特性，而且实现了对数据的有序管理。实验过程中，我亲眼见证了堆排序的强大功能，它不仅验证了堆不仅可以作为优先队列的数据结构，还可以作为高效的排序算法大显神威。此外，我还通过异常测试，确保了堆的实现能够在面对异常情况时依然保持稳健，妥善处理错误。

在这次实践中，我采用了面向对象的设计思路，特别是利用继承和多态的特性，使得我们的堆实现具备了出色的扩展性和维护性。此次实验不仅深化了我对堆结构及其应用的认知，还极大地提升了我的编程技巧和算法设计能力，让我在实践中不断成长。